



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 3: Integración de Capas en Spring Boot, Persistencia con SQL Server y Caso de Estudio VideoRent

Resumen

Esta práctica guiada tiene como objetivo guiar al estudiantado en el desarrollo de una solución empresarial full-stack integrada utilizando **Spring Boot** y **Microsoft SQL Server Developer Edition**. A lo largo de esta práctica, usted diseñará la base de datos y tablas utilizando **SQL Server Management Studio (SSMS)** basándose en el diagrama de entidad-relación de **VideoRent**.

Posteriormente, configurará la conexión JDBC en Spring Boot adaptando la versión de Java a su entorno local, y mapeará relaciones de tipo Many-to-One y Many-to-Many con tablas intermedias utilizando **Spring Data JPA**. Para finalizar, implementará servicios de negocio con lógica transaccional, inyección por constructor y expondrá endpoints de red REST autogenerando documentación técnica con **OpenAPI/Swagger UI**.

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java (cualquier versión instalada localmente), SQL Server Developer Edition, SQL Server Management Studio (SSMS), Maven.

Metas de Aprendizaje:

1. Recrear el esquema físico de base de datos de VideoRent (Pelicula, Genero, Actor, PeliculaActor) en SSMS.
2. Configurar la conexión JDBC de Spring Boot adaptando el `pom.xml` a la versión exacta del SDK de Java instalada en su computadora.
3. Implementar el mapeo objeto-relacional (JPA) para relaciones Many-to-One y Many-to-Many con tabla intermedia.
4. Desarrollar lógica transaccional de negocio para registrar películas con validación de existencia de género y actores, controlando excepciones de rollback.

Conceptos Clave (El 'Qué')

1. Relaciones Relacionales en JPA

En bases de datos relacionales, los datos se conectan mediante llaves foráneas. JPA permite mapear estas relaciones directamente a colecciones u objetos de Java: * **Relación Many-to-One (@ManyToOne):** Muchas películas pertenecen a un único género. Se asocia mediante `@JoinColumn` indicando la llave foránea física (`genero_id`). * **Relación Many-to-Many (@ManyToMany):** Una película puede tener múltiples actores, y un actor puede participar en múltiples películas. Se resuelve mediante la tabla intermedia `PeliculaActor` mapeada en Java con la anotación `@JoinTable`.

2. Configuración Dinámica de Java en Maven

Cada estudiante puede poseer una versión diferente del kit de desarrollo (JDK) en su computadora (ej. Java 17, 21, etc.). En esta práctica, aprenderá a parametrizar el archivo `pom.xml` de Maven para

compilar utilizando la versión de Java local correspondiente a su entorno, evitando fallos de incompatibilidad.

Guía de Base de Datos (SSMS Script)

Antes de iniciar con el desarrollo en Spring Boot, debe preparar la base de datos local en Microsoft SQL Server.

1. Abra la herramienta SQL Server Management Studio (SSMS) y conéctese a su motor local.
2. Abra una nueva ventana de consultas (*New Query*) y ejecute el siguiente script para crear la base de datos de VideoRent, las restricciones de integridad relacional e insertar semillas de prueba:

```
-- 1. Crear base de datos de VideoRent utilizando su carnet
CREATE DATABASE VideoRent[Carné]_II2026;
GO

USE VideoRent[Carné]_II2026;
GO

-- 2. Crear tabla Genero con clave primaria auto-incremental (IDENTITY)
CREATE TABLE Genero (
    genero_id INT IDENTITY(1,1) PRIMARY KEY,
    nombre_genero VARCHAR(50) NOT NULL
);

-- 3. Crear tabla Actor con clave primaria auto-incremental (IDENTITY)
CREATE TABLE Actor (
    actor_id INT IDENTITY(1,1) PRIMARY KEY,
    nombre_actor VARCHAR(30) NOT NULL,
    apellidos_actor VARCHAR(40) NOT NULL
);

-- 4. Crear tabla Pelicula con FK a Genero y PK auto-incremental (IDENTITY)
CREATE TABLE Pelicula (
    pelicula_id INT IDENTITY(1,1) PRIMARY KEY,
    titulo VARCHAR(50) NOT NULL,
    subtitulada BIT NOT NULL,
    estreno BIT NOT NULL,
    genero_id INT NOT NULL,
    -- Definición de la relación FK entre Pelicula y Genero
```

```

        CONSTRAINT FK_Pelicula_Genero FOREIGN KEY (genero_id)
            REFERENCES Genero(genero_id)
    );

-- 5. Crear tabla intermedia PeliculaActor con FKs a Pelicula y Actor
CREATE TABLE PeliculaActor (
    pelicula_id INT NOT NULL,
    actor_id INT NOT NULL,
    PRIMARY KEY (pelicula_id, actor_id),
    -- Definición de la relación FK entre PeliculaActor y Pelicula
    CONSTRAINT FK_PeliculaActor_Pelicula FOREIGN KEY (pelicula_id)
        REFERENCES Pelicula(pelicula_id) ON DELETE CASCADE,
    -- Definición de la relación FK entre PeliculaActor y Actor
    CONSTRAINT FK_PeliculaActor_Actor FOREIGN KEY (actor_id)
        REFERENCES Actor(actor_id)
);

-- 6. Semillas de datos iniciales
INSERT INTO Genero (nombre_genero)
VALUES ('Acción'), ('Ciencia Ficción'), ('Comedia');

INSERT INTO Actor (nombre_actor, apellidos_actor)
VALUES
    ('Keanu', 'Reeves'),
    ('Laurence', 'Fishburne'),
    ('Carrie-Anne', 'Moss');

```

Parte 1: Práctica Guiada (Paso a Paso)

Reemplace de forma obligatoria la sección `<carnet>` por su carnet universitario en minúsculas en las declaraciones de paquetes (ej: `cr.ac.ucr.paraíso.ie.c01234.practica3`).

1. Configuración de Dependencias y Versión de Java (pom.xml):

Abra su archivo `pom.xml` e incorpore las dependencias de JPA, SQL Server y OpenAPI. Preste atención a la sección de `<properties>` y configure el número de versión según el JDK instalado localmente en su computadora:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```

    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.2</version>
    <relativePath/>
</parent>

<groupId>cr.ac.ucr.paraíso.ie.carnet.practica3</groupId>
<artifactId>practica-3</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
    <!-- NOTA IMPORTANTE: Sustituya el valor '21' por la -->
    <!-- versión exacta de Java instalada en su computadora -->
    <!-- (por ejemplo: '17', '21', etc.) -->
    <java.version>21</java.version>
    <maven.compiler.source>${java.version}</maven.compiler.source>
    <maven.compiler.target>${java.version}</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.microsoft.sqlserver</groupId>
        <artifactId>mssql-jdbc</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.5.0</version>
    </dependency>
</dependencies>
</project>

```

2. Configuración de Propiedades (application.properties):

Modifique el archivo `src/main/resources/application.properties` para definir la conexión al servidor local de SQL Server, modificando la contraseña del usuario `sa` por la suya propia:

```
# Conexión JDBC a SQL Server (Reemplace [Carné] por su carnet)
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=VideoRent[Carné]_II2026;encrypt=
spring.datasource.username=sa
spring.datasource.password=SuPasswordSeguro
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

# Hibernate Config
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.SQLServerDialect
```

3. Mapeo de Entidades JPA (Capa domain):

Cree las tres clases de entidad correspondientes a la estructura de VideoRent.

- Cree `Genero.java` en:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/Genero.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "Genero")
public class Genero {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "genero_id")
    private Integer id;

    @Column(name = "nombre_genero", nullable = false, length = 50)
    private String nombre;

    public Genero() {}

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
```

```

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}

```

- Cree `Actor.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/Actor.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "Actor")
public class Actor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "actor_id")
    private Integer id;

    @Column(name = "nombre_actor", nullable = false, length = 30)
    private String nombre;

    @Column(name = "apellidos_actor", nullable = false, length = 40)
    private String apellidos;

    public Actor() {}

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getApellidos() { return apellidos; }
    public void setApellidos(String apellidos) { this.apellidos = apellidos; }
}

```

- Cree `Pelicula.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/Pelicula.java`

```

package cr.ac.ucr.paraíso.ie.carnet.practica3.domain;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "Pelicula")
public class Pelicula {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "pelicula_id")
    private Integer id;

    @Column(nullable = false, length = 50)
    private String titulo;

    @Column(nullable = false)
    private boolean subtitulada;

    @Column(nullable = false)
    private boolean estreno;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "genero_id", nullable = false)
    private Genero genero;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "PeliculaActor",
        joinColumns = @JoinColumn(name = "pelicula_id"),
        inverseJoinColumns = @JoinColumn(name = "actor_id")
    )
    private List<Actor> actores = new ArrayList<>();

    public Pelicula() {}

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public boolean isSubtitulada() { return subtitulada; }
    public void setSubtitulada(boolean sub) { this.subtitulada = sub; }

```

```

    public boolean isEstreno() { return estreno; }
    public void setEstreno(boolean est) { this.estreno = est; }

    public Genero getGenero() { return genero; }
    public void setGenero(Genero genero) { this.genero = genero; }

    public List<Actor> getActores() { return actores; }
    public void setActores(List<Actor> actores) { this.actores = actores; }
}

```

4. Creación de Repositorios (Capa data):

Cree las interfaces en el paquete `data`.

- Cree `PeliculaRepository.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/data/PeliculaRepository.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.data;

import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Pelicula;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface PeliculaRepository
    extends JpaRepository<Pelicula, Integer> {
}

```

- Cree `GeneroRepository.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/data/GeneroRepository.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.data;

import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Genero;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface GeneroRepository
    extends JpaRepository<Genero, Integer> {
}

```

```
}
```

- Cree `ActorRepository.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/data/ActorRepository.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.data;

import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Actor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ActorRepository
    extends JpaRepository<Actor, Integer> {
}
```

5. Excepción de Negocio y Servicio Transaccional (Capa business):

Defina la lógica y reglas transaccionales.

- Cree la excepción `VideoRentException.java` en:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/business/VideoRentException.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.business;

public class VideoRentException extends RuntimeException {
    public VideoRentException(String mensaje) {
        super(mensaje);
    }
}
```

- Cree el objeto de transferencia `PeliculaCreationDTO.java` en el paquete `dto`:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/dto/PeliculaCreationDTO.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.dto;

import java.util.List;
```

```

public class PeliculaCreationDTO {
    private String titulo;
    private boolean subtitulada;
    private boolean estreno;
    private Integer generoId;
    private List<Integer> actorIds;

    public PeliculaCreationDTO() {}

    public String getTitulo() { return titulo; }
    public void setTitulo(String t) { this.titulo = t; }

    public boolean isSubtitulada() { return subtitulada; }
    public void setSubtitulada(boolean s) { this.subtitulada = s; }

    public boolean isEstreno() { return estreno; }
    public void setEstreno(boolean e) { this.estreno = e; }

    public Integer getGeneroId() { return generoId; }
    public void setGeneroId(Integer gId) { this.generoId = gId; }

    public List<Integer> getActorIds() { return actorIds; }
    public void setActorIds(List<Integer> aIds) { this.actorIds = aIds; }
}

```

- Cree el servicio `PeliculaService.java` en el paquete `business`:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/business/PeliculaService.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.business;

import cr.ac.ucr.paraiso.ie.carnet.practica3.data.PeliculaRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica3.data.GeneroRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica3.data.ActorRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Pelicula;
import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Genero;
import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Actor;
import cr.ac.ucr.paraiso.ie.carnet.practica3.dto.PeliculaCreationDTO;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.ArrayList;
import java.util.List;

@Service

```

```

public class PeliculaService {

    private final PeliculaRepository peliculaRepository;
    private final GeneroRepository generoRepository;
    private final ActorRepository actorRepository;

    // Inyección por Constructor
    public PeliculaService(
        PeliculaRepository peliculaRepository,
        GeneroRepository generoRepository,
        ActorRepository actorRepository) {
        this.peliculaRepository = peliculaRepository;
        this.generoRepository = generoRepository;
        this.actorRepository = actorRepository;
    }

    @Transactional(rollbackFor = VideoRentException.class)
    public Pelicula registrarPelicula(PeliculaCreationDTO dto) {
        // 1. Validar que el género exista
        Genero genero = generoRepository.findById(dto.getGeneroId())
            .orElseThrow(() -> new VideoRentException(
                "El género con ID " + dto.getGeneroId()
                + " no existe."));

        // 2. Validar que se provean actores
        if (dto.getActorIds() == null || dto.getActorIds().isEmpty()) {
            throw new VideoRentException(
                "Debe registrar al menos un actor en la película.");
        }

        // 3. Buscar y validar cada actor en la base de datos
        List<Actor> actoresAsociados = new ArrayList<>();
        for (Integer actorId : dto.getActorIds()) {
            Actor actor = actorRepository.findById(actorId)
                .orElseThrow(() -> new VideoRentException(
                    "El actor con ID " + actorId
                    + " no existe."));
            actoresAsociados.add(actor);
        }

        // 4. Mapear y guardar la entidad
        Pelicula pelicula = new Pelicula();
        pelicula.setTitulo(dto.getTitulo());
        pelicula.setSubtitulada(dto.isSubtitulada());
        pelicula.setEstreno(dto.isEstreno());
        pelicula.setGenero(genero);
    }
}

```

```

        pelicula.setActores(actoresAsociados);
    }

    return peliculaRepository.save(pelicula);
}
}

```

6. Controlador REST y OpenAPI (Capa controller):

Cree la capa expuesta al cliente.

- Cree el archivo `PeliculaController.java` en el paquete `controller`:
`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/controller/PeliculaController.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.controller;

import cr.ac.ucr.paraiso.ie.carnet.practica3.business.PeliculaService;
import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Pelicula;
import cr.ac.ucr.paraiso.ie.carnet.practica3.dto.PeliculaCreationDTO;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/peliculas")
@Tag(name = "Películas", description = "Endpoints para la gestión de películas")
public class PeliculaController {

    private final PeliculaService service;

    public PeliculaController(PeliculaService service) {
        this.service = service;
    }

    @PostMapping
    @Operation(
        summary = "Registrar una nueva película",
        description = "Crea una película y la asocia a un género y actores."
    )
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Película creada"),

```

```

        @ApiResponse(responseCode = "400", description = "Error en validación")
    })
    public ResponseEntity<?> registrar(
        @RequestBody PeliculaCreationDTO dto) {
        try {
            Pelicula guardada = service.registrarPelicula(dto);
            return ResponseEntity.status(HttpStatus.CREATED).body(guardada);
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}

```

Parte 2: Taller de Depuración (El Error Común)

El objetivo de este taller es identificar cómo la **falta de configuración transaccional ante checked exceptions** rompe la consistencia relacional del esquema de base de datos.

Paso 1: Provocar el Fallo Transaccional

- Modifique temporalmente su método en `PeliculaService.java` para simular que ocurre un fallo físico de escritura al escribir logs en disco tras insertar la película, utilizando una excepción de tipo marcada (`java.io.IOException`):

```

@Transactional
public Pelicula registrarConFallo(PeliculaCreationDTO dto)
    throws Exception {
    Genero genero = generoRepository.findById(dto.getGeneroId()).orElseThrow();

    // Inserción en tabla Pelicula
    Pelicula p = new Pelicula();
    p.setTitulo(dto.getTitulo());
    p.setSubtitulada(dto.isSubtitulada());
    p.setEstreno(dto.isEstreno());
    p.setGenero(genero);
    Pelicula guardada = peliculaRepository.save(p);

    // Simular error al procesar el resto de dependencias (Ej: guardar actores)
    if (true) {

```

```
        throw new java.io.IOException("Error de disco duro.");
    }

    return guardada;
}
```

Paso 2: Ejecución y Comprobación en SSMS

1. Lance su servidor ejecutando en terminal `mvn spring-boot:run`.
2. Abra `http://localhost:8080/swagger-ui/index.html` y ejecute una petición para registrar la película. Verá que la petición responde con código 500 debido al error.
3. Abra SQL Server Management Studio (SSMS). En una consulta SQL, ejecute:

```
SELECT * FROM Pelicula;
SELECT * FROM PeliculaActor;
```

4. Analice el resultado: Observará que la película **sí se insertó físicamente** en la tabla `Pelicula`, pero la tabla asociativa `PeliculaActor` quedó vacía. Dado que `IOException` es una excepción marcada, Spring no ejecutó el rollback por defecto, provocando un dato huérfano sin su consistencia.

Paso 3: Resolución del Error

Cambie la anotación del servicio agregando la política explícita para toda excepción:

```
@Transactional(rollbackFor = Exception.class)
```

Vuelva a ejecutar la prueba y valide en SSMS que la base de datos revierte y no inserta ningún registro parcial si el método falla.

Parte 3: Reto Autónomo (Sin Solución)

Debe implementar de forma autónoma el proceso de **Asociación de un Actor a una Película**.

Requerimientos:

- Cree un endpoint de tipo POST en PeliculaController:
`POST /api/peliculas/{peliculaId}/actores/{actorId}`
- En la capa de negocio (PeliculaService), implemente un método transaccional:
`public void asociarActor(Integer peliculaId, Integer actorId)`
- El método debe validar:
 1. Que la película exista en base de datos.
 2. Que el actor exista en base de datos.
 3. Que el actor **no se encuentre previamente asociado** a dicha película (para evitar llaves primarias duplicadas en PeliculaActor).
- Modifique las colecciones JPA de la entidad en memoria y guarde los cambios.
- Compruebe el éxito del proceso ejecutando una consulta en SSMS sobre la tabla PeliculaActor y validando que el registro se guarde correctamente.